

Pocket Smart AI – Links

Team Leader: I. Sabana Fathima

1. Topic

Links

Epic: Prerequisites

2. Project Overview

Pocket Smart AI is an AI-powered application built with FastAPI that uses Google Gemini AI. Before development starts, the team needs the essential tools and reference links in one place.

3. Objective

To collect and share all the prerequisite links (tools, documentation, and accounts) that the team needs to set up and build the project.

4. Work Steps

- 1 Collect the download link for Python and install it.
- 2 Collect the FastAPI and Uvicorn documentation links.
- 3 Collect the Jinja2 templates documentation link.
- 4 Collect the Google AI Studio link (aistudio.google.com) for the Gemini API key.
- 5 Collect the Git and code editor (VS Code) download links.
- 6 Check that every link opens correctly.
- 7 Share the final list of links with the team leader and the team.

5. Main Parts of the Links

- Python and Uvicorn
- FastAPI and Jinja2 documentation
- Google AI Studio (Gemini API)
- Git and code editor

6. Expected Outcome

A checked list of prerequisite links that the whole team can use to prepare their systems and start working on the project.

7. Team Lead

Name: I. Sabana Fathima

Role: Team Lead

Topic: Links

Epic: Prerequisites

Pocket Smart AI – Workflow

Member Assignment: R. Irfana Mahmooda

Team Leader: I.Sabana Fathima

1. Topic

Workflow

Epic: Project Workflow

2. Project Overview

Pocket Smart AI is an AI-powered application built with FastAPI that uses Google Gemini AI. The workflow describes how the application runs from the user's first action to the final result.

3. Objective

To explain and document the complete project workflow, showing how each part of the application connects and in what order it runs.

4. Work Steps

- 1 The user opens the application and registers a new account, or logs in.
- 2 The backend checks the user details against the database.
- 3 After login, the user enters the required details in the form.
- 4 The FastAPI backend receives the request and validates the input.
- 5 The backend sends a prompt to Google Gemini AI using the API key.
- 6 Gemini AI generates the response and returns it to the backend.
- 7 The backend saves the result in the database.
- 8 The result is displayed to the user on the web page using templates.

5. Main Parts of the Workflow

- bullet User registration and login
- bullet Input form and validation
- bullet FastAPI backend processing
- bullet Gemini AI integration
- bullet Database storage
- bullet Result display

6. Expected Outcome

A clear workflow document that shows the full flow of the application, from user input to the generated result, so the whole team understands how the project works.

7. Team Member

Name: R. Irfana Mahmooda

Role: Team Member

Topic: Workflow

Epic: Project Workflow

Pocket Smart AI – Gemini API Setup

Member Assignment: M. Aysha Siddikka

Team Leader: I. Sabana Fathima

1. Topic

Set up Google account and enable Gemini API

Epic: Epic 1: Gemini AI initialization

2. Project Overview

FitBuddy is an AI-powered fitness plan generator that uses Google Gemini AI to create personalized 7-day workout plans based on user fitness details.

3. Objective

To set up the Google account, generate a Gemini API key, and confirm it works, so the FitBuddy application can connect to Gemini AI.

4. Work Steps

- 1 Create or sign in to a Google account at accounts.google.com (use a shared team account for the project).
- 2 Open Google AI Studio (aistudio.google.com) and accept the terms.
- 3 Click Get API key, then Create API key, and choose a new or existing Google Cloud project.
- 4 Store the key securely in an environment variable named `GEMINI_API_KEY`.
- 5 Make sure the key is never committed to Git (add the `.env` file to `.gitignore`).
- 6 Send a test request to the Gemini API and check that a valid response is returned.
- 7 Share the working setup with the team lead for use in the app.

5. Main Parts of the Setup

- Google account creation
- Google AI Studio access
- API key generation
- Secure key storage
- Test request and verification

6. Expected Outcome

A working Google account and Gemini API key, stored securely, that returns a valid response to a test request and is ready for the FitBuddy app to use.

7. Team Member

Name: M. Aysha Siddikka

Role: Team Member

Topic: Set up Google account and enable Gemini API

Epic: Epic 1: Gemini AI initialization

Pocket Smart AI – Configure Access and Permissions

Member Assignment – M. Ruhana Parvin

Team Lead – I.Sabana Fathima

1. Topic

Configure access and permissions

Epic: Epic 1: Gemini AI Initialization

2. Project Overview

Pocket Smart AI is an AI-powered fitness plan generator that uses Google Gemini AI to create personalized 7-day workout plans based on user fitness details.

3. Objective

To configure the access and permissions required to use Google Gemini AI in Pocket Smart AI, so that the application can connect to Gemini securely and only authorized components can use the API key.

4. Work Steps

1. Sign in to Google AI Studio and create a project for Pocket Smart AI.
2. Generate a Gemini API key for the project.
3. Restrict the API key so it can only be used for the Gemini API.
4. Store the API key in a .env file instead of writing it in the code.
5. Add the .env file to .gitignore so the key is never uploaded to GitHub.
6. Give team members access only to the roles and permissions they need.
7. Load the key in the application from the environment and test that Gemini responds.

5. Main Parts of Access and Permissions

- Google account and project setup
- API key generation
- API key restrictions
- Environment variable storage (.env)
- Team access and role permissions
- Connection testing

6. Expected Outcome

Gemini AI is securely configured with the correct access and permissions, and the Pocket Smart AI application can connect to it without exposing the API key.

7. Team Member

Name: M. Ruhana Parvin

Role: Team Member

Topic: Configure access and permissions

Epic: Epic 1: Gemini AI Initialization

Pocket Smart AI – Validate Gemini API Connectivity

Member Assignment: K. Jasiya

Team Leader: I. Sabana Fathima

1. Topic

Validate Gemini API connectivity

Epic: Epic 1: Gemini AI initialization

2. Project Overview

Pocket Smart AI is an AI-powered application built with FastAPI that uses Google Gemini AI. Before building any feature on top of it, the connection to the Gemini API must be confirmed to be working.

3. Objective

To check that the Gemini API key is valid and that the application can send a request to Gemini AI and receive a correct response.

4. Work Steps

- 1 Make sure the Gemini API key is stored in the GEMINI_API_KEY environment variable.
- 2 Install the Google Gemini Python package and load the key in a small test script.
- 3 Send a simple test prompt, such as "Hello", to the Gemini model.
- 4 Check that a valid response text is returned.
- 5 Test the error cases: a missing key, a wrong key, and no internet connection.
- 6 Confirm that clear error messages are shown when a request fails.
- 7 Report the test result to the team lead.

5. Main Parts of the Validation

- API key loading from the environment

- Test script with a simple prompt

- Response check

- Error case testing

- Test result report

6. Expected Outcome

A confirmed working connection to the Gemini API, where a test request returns a valid response and errors are handled clearly, so the team can safely build the app features on it.

7. Team Member

Name: K. Jasiya

Role: Team Member

Topic: Validate Gemini API connectivity

Epic: Epic 1: Gemini AI initialization

Pocket Smart AI – FastAPI Initialization and Import Libraries

Team Leader: I. Sabana Fathima

1. Topic

FastAPI initialization and import libraries

Epic: Epic 2: Core Functionalities Development

2. Project Overview

Pocket Smart AI is an AI-powered application built with FastAPI that uses Google Gemini AI. The backend needs a properly created FastAPI app and all required libraries imported before any feature can be built.

3. Objective

To create the FastAPI application instance and import all the libraries the project needs, so the other backend parts (routes, database, and Gemini AI) can be built on top of it.

4. Work Steps

- 1 Install the required libraries (fastapi, uvicorn, jinja2, python-dotenv, google-generativeai, and the database library).
- 2 Create the main Python file for the application.
- 3 Import FastAPI and the other required modules (Request, Form, Jinja2Templates, StaticFiles, RedirectResponse).
- 4 Import the standard libraries (os) and python-dotenv to read environment variables.
- 5 Create the FastAPI app instance with the project title.
- 6 Run the file and check that all imports work without errors.

5. Main Parts of the Initialization

- Library installation
- Importing FastAPI and project modules
- Importing environment and Gemini libraries
- FastAPI app instance creation

6. Expected Outcome

A working FastAPI app instance with all required libraries imported correctly, ready for routes and other core functions to be added.

7. Team Lead

Name: I. Sabana Fathima

Role: Team Lead

Topic: FastAPI initialization and import libraries

Epic: Epic 2: Core Functionalities Development

Pocket Smart AI – Main Functionality Code Routings

Member Assignment: R. Irfana Mahmooda

Team Leader: I.Sabana Fathima

1. Topic

Main functionality code routings

Epic: Epic 2: Core Functionalities Development

2. Project Overview

Pocket Smart AI is an AI-powered application built with FastAPI that uses Google Gemini AI. Its main features are reached through routes that connect the web pages to the backend code.

3. Objective

To write the routes for the main functionality of the application, so each page and action is connected to the correct backend function.

4. Work Steps

- 1 Create the route file and import FastAPI's APIRouter and the required modules.
- 2 Add a GET route for the home page and render its template.
- 3 Add a GET route that displays the main input form.
- 4 Add a POST route that receives the form data and validates it.
- 5 Call the Gemini AI function from the route and get the generated result.
- 6 Save the result in the database and return it to the user.
- 7 Add a route to view the saved results or history.
- 8 Add error handling and test every route.

5. Main Parts of the Routings

- Home page route
- Input form route (GET and POST)
- Gemini AI call from the route
- Result display route
- History route
- Error handling

6. Expected Outcome

Working routes that connect all the main pages and functions, so a user can submit details, get an AI-generated result, and view it in the application.

7. Team Member

Name: R. Irfana Mahmooda

Role: Team Member

Topic: Main functionality code routings

Epic: Epic 2: Core Functionalities Development

Pocket Smart AI – Login and Register Page Routings

Member Assignment: M. Aysha Siddikka

Team Leader: I. Sabana Fathima

1. Topic

Login and Register page routings

Epic: Epic 2: Core Functionalities Development

2. Project Overview

Pocket Smart AI is an AI-powered application built with FastAPI that uses Google Gemini AI. Users must register and log in before they can use its features.

3. Objective

To create the routes that display and handle the Login and Register pages, so users can create an account and sign in to the application.

4. Work Steps

- 1 Create the login.html and register.html templates using Jinja2.
- 2 Add a GET route for /register that displays the registration page.
- 3 Add a POST route for /register that reads the form data, validates it, and saves the new user in the database.
- 4 Add a GET route for /login that displays the login page.
- 5 Add a POST route for /login that checks the username and password against the database.
- 6 Redirect the user to the home page after a successful login or registration.
- 7 Show clear error messages for invalid input, duplicate users, or wrong credentials.
- 8 Test the complete register and login flow in the browser.

5. Main Parts of the Routing

- Register page route (GET and POST)

- Login page route (GET and POST)

- Form validation

- Database user check

- Redirects and error messages

6. Expected Outcome

Working Login and Register pages where a new user can register, an existing user can log in, and errors are shown clearly when something goes wrong.

7. Team Member

Name: M. Aysha Siddikka

Role: Team Member

Topic: Login and Register page routings

Epic: Epic 2: Core Functionalities Development

Pocket Smart AI – Ensure Modular Code Structure

Member Assignment – M. Ruhana Parvin

Team Lead – I.Sabana Fathima

1. Topic

Ensure modular code structure

Epic: Epic 2: Core Functionalities Development

2. Project Overview

Pocket Smart AI is an AI-powered fitness plan generator that uses Google Gemini AI to create personalized 7-day workout plans based on user fitness details.

3. Objective

To organize the core functionalities of Pocket Smart AI into separate, reusable modules, so that the code is easy to read, test, maintain, and extend by all team members.

4. Work Steps

1. Plan the project folder structure and decide which files each module will have.
2. Create separate modules for workout, nutrition, feedback, and user-management functions.
3. Keep the Gemini AI functions in their own module, separate from the routes.
4. Move database models and connection code into a separate database module.
5. Keep each function small and focused on one task, with clear names and comments.
6. Import the modules into the main file instead of repeating code.
7. Test each module on its own, then test the complete workflow together.

5. Main Parts of Modular Structure

- Project folder and file organization
- Workout and nutrition modules
- Feedback and user-management modules
- Gemini AI module
- Database module
- Module testing and integration

6. Expected Outcome

A clean, well-organized code structure where each core function lives in its own module, making Pocket Smart AI easier to maintain and allowing team members to work in parallel.

7. Team Member

Name: M. Ruhana Parvin

Role: Team Member

Topic: Ensure modular code structure

Epic: Epic 2: Core Functionalities Development

Pocket Smart AI – Define FastAPI Routes

Member Assignment: K. Jasiya

Team Leader: I.Sabana Fathima

1. Topic

Define FastAPI routes

Epic: Epic 3: Backend - FastAPI Integration

2. Project Overview

Pocket Smart AI is an AI-powered application built with FastAPI that uses Google Gemini AI. The backend receives requests from the web pages through routes, so each route must be defined clearly.

3. Objective

To define the FastAPI routes that connect each web page and action to the correct backend function, using the right HTTP methods.

4. Work Steps

- 1 Plan the list of routes needed, such as home, login, register, main feature, and history.
- 2 Create the route file and import APIRouter and the required modules.
- 3 Define GET routes to display the pages using Jinja2 templates.
- 4 Define POST routes to receive and validate form data.
- 5 Connect each route to its backend function, including the Gemini AI call.
- 6 Add response models and proper status codes.
- 7 Include the router in the main app file.
- 8 Test all routes using the browser and the FastAPI /docs page.

5. Main Parts of the Routes

- bulletRoute planning

- bulletGET routes for pages

- bulletPOST routes for form data

- bulletConnection to backend functions

- bulletResponse models and status codes

- bulletRouter inclusion in the main app

6. Expected Outcome

A complete set of working FastAPI routes that connect every page and action to the backend, all visible and testable in the /docs page.

7. Team Member

Name: K. Jasiya

Role: Team Member

Topic: Define FastAPI routes

Epic: Epic 3: Backend - FastAPI Integration

Pocket Smart AI – Modular Architecture Setup

Team Leader: I. Sabana Fathima

1. Topic

Modular architecture setup

Epic: Epic 3: Backend - FastAPI Integration

2. Project Overview

Pocket Smart AI is an AI-powered application built with FastAPI that uses Google Gemini AI. To keep the backend clean and easy to maintain, the code is split into separate modules instead of one large file.

3. Objective

To set up a modular project structure with separate folders and files for routes, database, templates, static files, and Gemini AI logic, so the team can work on different parts without conflicts.

4. Work Steps

- 1 Plan the project structure and decide the folders and files needed.
- 2 Create the main file (main.py) as the application entry point.
- 3 Create a routes folder for the login, register, and other route files.
- 4 Create separate files for database setup and models.
- 5 Create a separate file for the Gemini AI service.
- 6 Create the templates and static folders for Jinja2 pages, CSS, and images.
- 7 Add a .env file for settings and keep it out of Git using .gitignore.
- 8 Connect all modules to the main file and check that the app runs without errors.

5. Main Parts of the Architecture

- Main entry file
- Routes module
- Database module
- Gemini AI service module
- Templates and static folders
- Environment settings

6. Expected Outcome

A clean modular project structure where each part of the backend has its own file or folder, and all modules connect correctly to the main FastAPI app.

7. Team Lead

Name: I. Sabana Fathima

Role: Team Lead

Topic: Modular architecture setup

Epic: Epic 3: Backend - FastAPI Integration

Pocket Smart AI – CORS & Static Routing

Member Assignment: R. Irfana Mahmooda

Team Leader: I. Sabana Fathima

1. Topic

CORS & Static Routing

Epic: Epic 3: FastAPI Integration

2. Project Overview

Pocket Smart AI is an AI-powered application built with FastAPI that uses Google Gemini AI. The backend must allow safe access from the browser and serve its static files, such as CSS, JavaScript, and images.

3. Objective

To configure CORS (Cross-Origin Resource Sharing) and static file routing in FastAPI, so the frontend can connect to the backend and load its static files correctly.

4. Work Steps

- 1 Import CORSMiddleware from FastAPI in the main file.
- 2 Add the CORS middleware to the app with the allowed origins, methods, and headers.
- 3 Create a static folder for the CSS, JavaScript, and image files.
- 4 Import StaticFiles and mount the static folder in the app at the /static path.
- 5 Link the static files in the HTML templates using the /static path.
- 6 Run the application and check that the static files load without errors.
- 7 Test that requests from the frontend are accepted and no CORS error appears in the browser.

5. Main Parts of the Setup

- CORS middleware configuration
- Allowed origins, methods, and headers
- Static folder creation
- Static files mounting
- Linking static files in templates

6. Expected Outcome

A FastAPI application with CORS configured and static routing working, so the pages load their CSS, JavaScript, and images correctly and frontend requests reach the backend without errors.

7. Team Member

Name: R. Irfana Mahmooda

Role: Team Member

Topic: CORS & Static Routing

Epic: Epic 3: FastAPI Integration

Pocket Smart AI – Startup and Main Function

Member Assignment: M. Aysha Siddikka

Team Leader: I. Sabana Fathima

1. Topic

Startup and main function

Epic: Epic 3: Backend - FastAPI Integration

2. Project Overview

Pocket Smart AI is an AI-powered application built with FastAPI that uses Google Gemini AI. The backend must start correctly and load all its components before the app can be used.

3. Objective

To write the startup code and main function that create the FastAPI application, load its settings, connect the routes, and run the server.

4. Work Steps

- 1 Create the main file and import FastAPI and the required project modules.
- 2 Create the FastAPI app instance with the project title.
- 3 Load environment variables, including the Gemini API key.
- 4 Connect the Jinja2 templates and static files.
- 5 Create the database tables when the application starts.
- 6 Include the route files (login, register, and other routes) in the main app.
- 7 Add a main function with a Uvicorn run command to start the server.
- 8 Run the application and check that it starts without errors.

5. Main Parts of the Startup

- FastAPI app creation
- Environment and API key loading
- Templates and static files setup
- Database initialization
- Router inclusion
- Uvicorn server start

6. Expected Outcome

A working main file that starts the FastAPI server, loads all components, and makes the application available in the browser.

7. Team Member

Name: M. Aysha Siddikka

Role: Team Member

Topic: Startup and main function

Epic: Epic 3: Backend - FastAPI Integration

Pocket Smart AI – Develop HTML Frontend Interface

Member Assignment – M. Ruhana Parvin
Team Lead – I. Sabana Fathima

1. Topic

Develop HTML frontend interface
Epic: Epic: UI Development

2. Project Overview

Pocket Smart AI is an AI-powered fitness plan generator that uses Google Gemini AI to create personalized 7-day workout plans based on user fitness details.

3. Objective

To design and build the HTML frontend pages of Pocket Smart AI, so that users can easily enter their fitness details and view their generated 7-day workout plan in a clean, responsive interface.

4. Work Steps

1. Plan the pages needed, such as the home page, user details form, and workout plan result page.
2. Create the HTML templates using Jinja2 with a common base layout.
3. Build the user input form for details like age, weight, height, fitness goal, and activity level.
4. Design the result page to display the 7-day workout plan clearly, day by day.
5. Add CSS styling in the static folder to make the interface clean and mobile-friendly.
6. Add basic form validation and clear messages for errors and loading.
7. Connect the templates to the FastAPI routes and test the complete user flow.

5. Main Parts of the Frontend

- Base template and page layout
- Home page
- User details input form
- Workout plan result page
- CSS styling and responsive design
- Form validation and error messages

6. Expected Outcome

A simple, attractive, and user-friendly HTML interface where users can enter their fitness details and see their personalized workout plan generated by Pocket Smart AI.

7. Team Member

Name: M. Ruhana Parvin

Role: Team Member

Topic: Develop HTML frontend interface

Epic: Epic: UI Development

Pocket Smart AI – Test with Real-World Budget Inputs

Member Assignment: K. Jasiya

Team Leader: I.Sabana Fathima

1. Topic

Test with real-world budget inputs across domains

Epic: Epic 5: Testing and Deployment

2. Project Overview

Pocket Smart AI is an AI-powered application built with FastAPI that uses Google Gemini AI. Before deployment, it must be tested with realistic budget inputs from different domains to confirm that it gives useful and correct results.

3. Objective

To test the application with real-world budget inputs across different domains, and confirm that it handles them correctly and produces sensible results.

4. Work Steps

- 1 List the domains to test, such as personal, household, student, travel, and small business budgets.
- 2 Prepare realistic test inputs for each domain, with low, medium, and high budget amounts.
- 3 Run each test input through the application and record the output.
- 4 Check that the results are relevant, correct, and suited to the budget given.
- 5 Test edge cases, such as zero, very large, negative, and non-numeric values.
- 6 Note any bugs, errors, or wrong outputs found during testing.
- 7 Share the issues with the team lead and re-test after the fixes.
- 8 Prepare a short test report with the results.

5. Main Parts of the Testing

- Domain list and test inputs
- Low, medium, and high budget cases
- Output verification
- Edge case testing
- Bug reporting and re-testing
- Test report

6. Expected Outcome

A tested application that gives correct and useful results for real-world budget inputs in every domain, with all found issues fixed and a test report ready before deployment.

7. Team Member

Name: K. Jasiya

Role: Team Member

Topic: Test with real-world budget inputs across domains

Epic: Epic 5: Testing and Deployment

Pocket Smart AI – Conclusion

Team Leader: I. Sabana Fathima

1. Topic

Conclusion

Epic: Conclusion

2. Project Overview

Pocket Smart AI is an AI-powered application built with FastAPI that uses Google Gemini AI. The project covered Gemini API setup, FastAPI initialization, modular architecture, login and register routing, and the startup and main function.

3. Objective

To summarize the work completed in the project, the results achieved, and the overall outcome of the Pocket Smart AI application.

4. Work Steps

- 1 Review all the epics and the work completed in each one.
- 2 Confirm that the Gemini API works and the key is stored securely.
- 3 Confirm that the FastAPI app, modular structure, and routes work together.
- 4 Test the complete flow: startup, register, login, and AI features.
- 5 Note the results and any improvements for the future.
- 6 Prepare the final summary for the team leader.

5. Main Parts of the Conclusion

- Summary of completed work
- Final testing results
- Key learnings
- Future improvements

6. Expected Outcome

A clear final summary showing that the Pocket Smart AI backend is complete, working, and ready for use, along with ideas for future improvement.

7. Team Lead

Name: I. Sabana Fathima

Role: Team Lead

Topic: Conclusion

Epic: Conclusion